

When Does Memory Buy Adaptation? Evaluating In-Context Reinforcement Learning Under Model-Selection and Real-Time Constraints

Felix Surjodinoto

Keandre Rafael

Delroy

Samuel Philip

Computer Science Department Computer Science Department Computer Science Department Computer Science Department
School of Computer Science School of Computer Science School of Computer Science School of Computer Science
Bina Nusantara University Bina Nusantara University Bina Nusantara University Bina Nusantara University
Jakarta 11530, Indonesia Jakarta 11530, Indonesia Jakarta 11530, Indonesia Jakarta 11530, Indonesia
felix.surjodinoto@binus.ac.id keandre.rafael@binus.ac.id delroy@binus.ac.id samuel.philip@binus.ac.id

Abstract—Real-time control denies reinforcement learning its usual luxuries: the simulator neither pauses while the policy computes nor runs faster than the wall clock. We study what it takes to obtain both *competence* and *rapid adaptation* under these constraints, comparing a reactive convolutional agent and causal-transformer RL² agents on a real-time racing task (20 Hz, TrackMania 2020) and a controlled continuous-control benchmark (MuJoCo). On the adaptation side, we evaluate whether in-context meta-RL trained exclusively on action-value corruptions can generalize to unobserved temporal actuator latency. We show that conclusions regarding cross-mechanism adaptation depend critically on checkpoint selection: while an unverified exploratory test-aware search suggested an apparent latency adaptation gap of $+652 \pm 225$ return, a strict confirmatory protocol across 8 independent training seeds ($N = 8$, selecting checkpoints solely on unseen parameter ranges of the training distribution) reveals that adaptation is seed-sensitive and unestablished, yielding a mean adaptation gap of -61.8 with a 95% seed-level bootstrap confidence interval spanning zero ($[-231.3, +141.6]$, with 6 of 8 seeds exhibiting negative adaptation). Furthermore, an independent reactive MLP baseline matches the transformer on latency ($+447.3$ [$404.0, 498.2$]) and decisively outperforms it on extreme faults ($+2470.8$ vs. $+1929.3$ on locked actuators). On the competence side, a demonstration-anchored reactive agent (behavior-cloning warm start from 23 human laps, a pinned demonstration pool in half of every batch, deterministic-episode collection) reaches deterministic track completion within one day of real-time training, whereas transformer agents fall short on identical demonstrations: first under a defective sequence replay, then under documented systems and distribution-coverage bottlenecks specific to sequence replay and shared trunks. While correcting replay coverage stabilized training, the sequence agent did not achieve track completion. We conclude that apparent in-context generalization across fault mechanisms is highly sensitive to model selection, and that in real-time continuous control, demonstrations reliably buy competence while interaction memory incurs heavy systems overhead without guaranteeing cross-mechanism adaptation.

Index Terms—Reinforcement learning, real-time control, in-context learning, meta-learning, learning from demonstration, sequence replay.

I. INTRODUCTION

Deep reinforcement learning has become a standard tool for continuous control, with off-policy actor-critic methods [1]–[3] and high-UTD ensemble variants [4]–[6] reaching strong sample efficiency on simulated benchmarks. Transferring this progress to physical vehicles is hard: training on hardware is unsafe and slow, and, more fundamentally, standard RL environments are *blocking*, meaning the simulator pauses while the policy computes, so the learned policy never experiences the execution latency, fixed control intervals, and concurrent time flow of a real control loop [7], [8]. TrackMania 2020 offers a useful middle ground: a high-fidelity driving simulator that cannot be paused, imposing real-time constraints identical in kind to a physical vehicle’s without the safety cost. We use it as the deployment target of our framework.

Deployed controllers face a further challenge that sample efficiency does not address: *faults*. An actuator degrades, locks, or starts responding with delay, and the controller must compensate online, without gradient updates. In-context meta-RL in the RL² family [9]–[11] is a natural candidate, since a sequence-model policy can infer the active perturbation from its own action-consequence history. However, existing evaluations typically test adaptation to unseen *parameters* of training task families; whether in-context adaptation transfers to a structurally distinct corruption class—specifically generalizing from value-magnitude perturbations to temporal actuator latency without parameter updates—remains an open question. Crucially, a reported return under a held-out fault can be distorted by test-split checkpoint leakage or by a robust memoryless policy. Disentangling genuine in-context adaptation from these confounders requires a leak-free validation selection protocol, an ablation that removes context while preserving trained weights, and an independently trained reactive robust baseline.

In-context capability, however, is only half of the question for a deployed system; the other half is whether the machinery that provides it survives real-time training at all. We therefore

examine three agents on the same racing task: a transformer RL^2 agent with a defective sequence replay (whose post-hoc diagnosis motivates the rest), a demonstration-anchored reactive convolutional agent, and a corrected transformer agent trained on identical demonstrations. The reactive agent reaches deterministic track completion within one day of real-time training; the sequence agents do not. Diagnosing sequence replay and shared-trunk training mechanisms resolves several deterministic failure modes, but the reactive agent completes the task whereas the sequence agent does not, highlighting the substantial systems costs of sequence modeling in real-time control.

The contributions of this paper are:

- **A controlled fault-adaptation study with a clean validation-only protocol:** training on action-value corruptions only, we contrast unverified exploratory test-aware checkpoint selection against strict validation selection across 8 independent training seeds ($N = 8$). Under confirmatory validation selection, the adaptive transformer yields an adaptation gap spanning zero ($-61.8 [-231.3, +141.6]$) on unobserved temporal latency with pronounced seed sensitivity (6 of 8 seeds exhibiting negative adaptation), while an independent reactive robust baseline matches its latency return ($+447.3 [404.0, 498.2]$) and surpasses it on extreme actuator faults, demonstrating that the large exploratory adaptation gap ($+652 \pm 225$) was driven by checkpoint selection feedback and that cross-mechanism latency adaptation is seed-sensitive and cannot be assumed without temporal supervision.
- **A demonstration-anchored recipe for real-time competence:** behavior-cloning warm start from 23 human laps (≈ 30 minutes of demonstrator time), an RLPD-style pinned demonstration pool filling half of every batch [12], random-shift image augmentation [13], layer-normalized critics [5], and collection of deterministic-policy episodes into the replay buffer. The resulting reactive agent completes the track deterministically within one day of 20 Hz real-time training.
- **A deployment-cost study of in-context agents:** six failure mechanisms specific to sequence replay and shared-trunk training, each diagnosed from training telemetry and mitigated (terminal-inclusive variable-length windows, episode-start oversampling, critic-warmup feature freezing, and a behavior-cloning anchor that maintains trunk and encoder features rather than only the policy head).
- **A stabilized transformer RL^2 deployment architecture:** a causal-transformer context encoder trained by sequence-level REDQ-SAC with layer-normalized critic heads [5] and a detached actor head, evaluated alongside exploratory continuous-time latent blocks and a RESeL-style context-encoder learning-rate split [15], on a distributed real-time architecture with a 20 Hz control loop enforced by `rtgym` [16], [17].

We do not claim a new algorithm: the agents are assembled

from established components, and we make no state-of-the-art comparison against systems such as AMAGO [11]. The scientific contribution is the mechanism-level transfer question and the ablation and baseline that answer it, the demonstration-anchored real-time recipe, and an honest, mechanism-level account of what in-context machinery costs in a deployed real-time system.

II. RELATED WORK

A. Off-Policy RL and Sample Efficiency

Off-policy actor-critic methods [1], [2] and maximum-entropy SAC [3] dominate continuous control. Raising the update-to-data (UTD) ratio improves sample efficiency but destabilizes value learning. REDQ [4] maintains an ensemble of N critics and computes Bellman targets from the minimum over a random subset of size M ; DroQ [5] achieves similar control with dropout and layer normalization inside the critics; CrossQ [6] removes target networks via batch normalization. Distributed actor-learner systems [18] decouple collection from optimization, which our architecture adopts in a real-time setting.

B. Real-Time RL and Delayed MDPs

Real-time RL [7] formalizes the setting where the environment advances while the agent computes; delayed-MDP theory [8] and random-delay methods [19] treat action/observation latency explicitly. Real-Time Gym (`rtgym`) [16] enforces fixed control intervals: observations are captured at interval boundaries, actions are applied exactly one interval later, and computation overruns are handled elastically instead of pausing the simulator. Our control loop builds directly on it, and action latency reappears in our benchmark as the held-out cross-mechanism fault.

C. In-Context Meta-RL, Fault Adaptation, and Neural ODEs

RL^2 [9], [10] trains recurrent policies that implement a learned RL procedure in their hidden-state dynamics; PEARL [20] infers an explicit probabilistic task variable off-policy; Decision Transformer [21] and AMAGO [11] replace the RNN with transformers. All evaluate held-out *parameters* of training task families. For fault recovery specifically, GrBAL [22] and FAMLE [23] meta-learn dynamics models that adapt online to perturbed dynamics drawn from the training family, and domain randomization [24] trains policies robust to wide distributions, which is exactly the explanation our no-history ablation isolates. Neural ODEs [14] model hidden states as continuous-time dynamical systems $dh/d\tau = f(h)$, a natural fit for latents that must remain consistent under continuous time flow and actuation latency.

D. Learning from Demonstrations

Demonstrations address precisely the resource that real-time RL lacks: exploration paid in wall-clock time. DQfD [25] pretrains on demonstrations and keeps them in replay with an imitation margin loss; RLPD [12] shows that simply composing half of every batch from a pinned offline buffer,

together with layer-normalized critics and high update ratios, matches more elaborate offline-to-online methods; random-shift augmentation [13] is the standard regularizer for pixel-based actor-critic learning. Our reactive deployment recipe combines these three with a behavior-cloning warm start, and our sequence agent extends the anchor idea: when the actor head reads a shared trunk through a stop-gradient, an imitation loss confined to the head cannot prevent the critic from repurposing the features beneath it, so the anchor must backpropagate into the encoder.

E. Recurrent and Transformer Stability in Off-Policy RL

Sequence replay has its own failure modes, first catalogued for recurrent value functions by R2D2 [26]: stored-state staleness, burn-in, and the mismatch between the state distributions of training windows and deployment. Our deployment study adds image-scale variants of this catalogue, including a starvation effect in which uniform window sampling almost never trains the short-context episode-start states the deployed policy must handle after every reset. When critics share a recurrent or transformer trunk, high-UTD critic gradients additionally dominate the shared representation, causing oversaturation and catastrophic forgetting. RESeL [15] shows that recurrent off-policy RL requires a context-encoder-specific (lower) learning rate. We adopt this two-group optimization in the real-time deployment configuration, and in the controlled benchmark we report direct evidence that layer-normalized critic heads were necessary in our setting, where ensemble critics sharing one trunk are highly correlated and min-based pessimism is weakened.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Distributed Architecture and Real-Time Control Loop

TMRL adopts a decoupled client-server architecture: rollout workers co-located with the TrackMania 2020 simulator execute the current policy and buffer transitions; a central broker relays data and weights over the `tlspyo` library [27]; a GPU trainer aggregates experience, performs the gradient updates, and periodically broadcasts new weights. The control loop is enforced by Real-Time Gym (`rtgym`) [16] at a fixed 20 Hz ($\Delta t = 0.05$ s): the observation is captured at the interval’s start boundary, the action is applied via a virtual gamepad exactly at the end boundary, and computation overruns are handled elastically instead of pausing the simulator.

B. Action and Observation Spaces

The action is $a_t = [g_t, b_t, s_t] \in [-1, 1]^3$ (gas, brake, steering). Observations are grayscale screenshots resized to 64×64 (the reactive agent stacks 4 frames; the sequence agents use a single frame per step, the transformer history supplying temporal context), telemetry (speed, gear, engine RPM) retrieved via the OpenPlanet API [28], and the previously applied actions.

C. Transformer RL^2 Trunk and Task-Inference Head

For in-context adaptation, each timestep is embedded as one token: a linear *timestep embedder* projects $[s_t \parallel a_{t-1} \parallel r_{t-1} \parallel d_{t-1}]$ to $d_{\text{model}} = 64$, where d is an episode-boundary indicator; in the visual configuration a per-timestep CNN first encodes the image stack to a 128-dimensional feature. The token sequence, plus learned positional embeddings, passes through a 2-layer pre-norm causal transformer (2 heads, feed-forward width 128, dropout 0.1, GELU, context window up to 128 tokens), yielding per-token hidden states $h_t \in \mathbb{R}^{64}$. At inference the actor keeps a deque of recent timesteps and re-runs the forward pass each step (negligible cost at this scale); the deque is cleared on episode boundaries, so context never leaks across episodes.

A small *task encoder* maps each h_t to an 8-dimensional task embedding z_t , which conditions both actor and critic heads, and to classification logits over the fault taxonomy, trained with an auxiliary cross-entropy loss against the episode’s ground-truth fault label (training-time only; weight $\lambda_{\text{task}} = 0.05$). This gives the trunk an explicit incentive to encode fault identity rather than relying solely on the return objective.

D. Continuous-Time Latent Dynamics via Neural ODEs

The output hidden state of the causal transformer trunk is modeled as a continuous-time dynamical system:

$$\frac{dh(\tau)}{d\tau} = f(h(\tau), \theta_{\text{ODE}}) \quad (1)$$

where f is a multi-layer perceptron consisting of Layer Normalization, a linear layer, GELU activation, and a final linear layer. To stabilize the start of training, the final layer’s weights are initialized to zero, making $f(h, \theta_{\text{ODE}}) \approx 0$ initially, so the block begins as the identity.

We integrate the latent state over a virtual duration $\Delta\tau$ (`ode_dt` = 1.0) using K fixed steps $\delta\tau = \Delta\tau/K$ with either the explicit Euler scheme, $h_{k+1} = h_k + \delta\tau f(h_k, \theta_{\text{ODE}})$, or the classical fourth-order Runge–Kutta scheme; the final integrated state h_K is passed to the policy and critic heads. This block targets the latency-afflicted real-time setting; the controlled benchmark experiments disable it, keeping the headline result attributable to the context mechanism alone.

E. Stabilized Off-Policy Sequence Training

The learning algorithm is a sequence-level adaptation of REDQ-SAC [3], [4]: an ensemble of $N = 10$ critic heads shares the transformer trunk, and Bellman targets use the minimum over a random subset of $M = 2$ target critics with entropy-regularized SAC targets and a learned temperature. Replay stores whole trajectories; each gradient step samples windows of $L_{\text{burn}} + L_{\text{train}}$ consecutive timesteps ($8 + 24$ in the controlled benchmark, $20 + 64$ in the TrackMania configuration), where the first L_{burn} positions condition the trunk but are masked out of all losses. The episode-boundary flag is terminated-only (truncation does not set it), consistent with the terminated-only bootstrap in the sequence memory. Throughout this paper, UTD denotes

the number of critic updates per policy update (the implementation’s `q_updates_per_policy_update`), not gradient steps per environment step.

At high UTD the shared trunk receives many more critic gradients than actor gradients, leading to representation over-saturation and catastrophic forgetting as the buffer saturates. Two measures protect the trunk in all configurations: the actor head reads trunk features through a stop-gradient (at UTD 10–15 critic gradients outnumber actor gradients by an order of magnitude), and each critic head is a 2-hidden-layer MLP with LayerNorm after every hidden linear layer [5], which proved essential in practice. The real-time configuration additionally follows RESeL [15], dividing parameters into two optimization groups, the context encoder (transformer trunk, embedder, and Neural ODE) and the task heads (policy and critics), with a reduced context learning rate

$$\eta_{\text{context}} = \gamma_{\text{RESeL}} \cdot \eta_{\text{heads}}, \quad (2)$$

where $\gamma_{\text{RESeL}} = 0.1$ by default and η_{context} may instead be set explicitly (our deployment run uses $\eta_{\text{context}} = 5 \times 10^{-5}$). The controlled benchmark runs do not enable this split: there, the context encoder trains at the full critic learning rate, so the benchmark results isolate the contribution of the context mechanism without RESeL.

F. Demonstration-Anchored Real-Time Training

Because the simulator runs strictly in real time, exploration is bounded by the wall clock: at 20 Hz, a day of training yields at most 1.7×10^6 transitions regardless of compute. We therefore anchor deployment training on human demonstrations, recorded by replacing the policy with a reader of the demonstrator’s input device while routing observations, rewards, and resets through the unmodified pipeline (so recorded transitions are distributionally identical to agent experience; unfinished episodes are rejected). Twenty-three complete laps (26,642 transitions, ≈ 30 minutes of demonstrator time, best lap ≈ 54 s) anchor training three ways: a behavior-cloning warm start; an RLPD-style pinned demonstration pool [12] filling half of every batch and never evicted (which doubles as an anti-forgetting mechanism); and, for the sequence agent, a behavior-cloning anchor term in the actor loss. Additionally, every k -th training episode is collected *with the deterministic policy* and sent to replay, so deterministic failure modes become training data, and random-shift augmentation [13] regularizes the image pathway.

G. Terminal-Aware Sequence Replay

The deployment study traces most sequence-agent failures to the replay. The corrected design differs from a naive window sampler in four ways. *Terminal inclusion*: variable-length windows run up to and including the episode’s final step, so terminal rewards enter training; termination zeroes the bootstrap while truncation bootstraps from the real successor. *Short-episode support*: episodes shorter than the window are right-padded and masked instead of discarded. *Episode-start oversampling*: uniform offset sampling yields one episode-start

TABLE I
FAULT TAXONOMY. SEVERITY $s \in [0, 1]$; EXPERIMENTS USE $s = 1$. FAULTS ARE RESAMPLED EACH EPISODE AND ARE NOT OBSERVABLE.

Fault	Mechanism	Effect (per episode)
<code>action_scale</code>	value	each action dim. scaled by $u \sim \mathcal{U}[1-0.5s, 1+0.5s]$
<code>action_noise</code>	value	i.i.d. Gaussian noise, std. $0.5s$, added to actions
<code>sensor_noise</code>	value (obs.)	i.i.d. Gaussian noise, std. $0.5s$, added to observations
<code>dead_actuator</code>	value (extreme)	one random action dim. locked to 0
<code>sign_flip</code>	value (extreme)	one random action dim. negated
<code>action_latency</code>	temporal	executed action is the one commanded k steps ago, $k \sim \mathcal{U}\{1, \dots, 5\}$

window per $\sim 1,130$ mid-episode windows, starving the short-context launch behavior required after every reset (and starving behavior-cloning pretraining, which uses the same sampler); a fixed fraction (0.25) of windows is forced to begin at episode starts. *Feature-maintaining anchor*: the behavior-cloning anchor backpropagates through non-detached hidden states into the encoder at the slow context rate, while the value term still reaches only the head; without this, the critic, which alone trains the shared trunk and CNN, continuously repurposes the features under the actor head.

H. Fault-Injection Benchmark and the No-History Ablation

To evaluate in-context adaptation under control, a wrapper turns any flat-Box Gymnasium [29] environment into an RL² task: it packs $(a_{\text{prev}}, r_{\text{prev}}, d_{\text{prev}})$ into the observation tuple and draws one unobservable fault per episode from the taxonomy in table I. The stored a_{prev} is the *commanded* (pre-fault) action, so the agent observes what it asked for and then the corrupted consequence; that discrepancy is the meta-learning signal. Formally, the agent commands a_t and the environment executes $\tilde{a}_t = g_\phi(a_t, h_t)$ for episode fault ϕ , with $\text{supp}(p_{\text{eval}}) \cap \text{supp}(p_{\text{train}}) = \emptyset$ at test time, and the policy is the sequence model $a_t \sim \pi_\theta(\cdot \mid s_{\leq t}, a_{< t}, r_{< t}, d_{< t})$.

Training draws uniformly from the value-corruption faults `action_scale` and `action_noise`. Evaluation uses two disjoint held-out splits: **within-mechanism extrapolation** (`dead_actuator`, `sign_flip`: still per-dimension value corruption, but at gains of 0 and -1 when training never goes below 0.5) and **cross-mechanism transfer** (`action_latency`: action values preserved but delayed by k steps, a temporal corruption absent from training).

To attribute held-out performance correctly, every evaluation runs the identical weights twice: **adaptive** (the context deque accumulates the episode history) and **no-history** (the deque is cleared before every action selection, reducing the agent to a nearly memoryless reactive policy). The *adaptation gap* $G = R_{\text{adaptive}} - R_{\text{no-history}}$ measures exactly the contribution of in-context memory: a robust history-independent policy gives $G \approx 0$, whereas genuine in-context adaptation gives $G > 0$. The ablation costs one extra evaluation pass and no retraining.

IV. EXPERIMENTAL EVALUATION

A. Controlled Fault-Adaptation Experiments

The base environment is HalfCheetah-v5 (MuJoCo [30]; 17-dimensional observations, 6 actuators), with the Neural ODE block and the RESeL split disabled to isolate the sequence mechanism. Agents train for up to 10^3 iterations of $\{4 \text{ collection episodes} \rightarrow 100 \text{ gradient steps}\}$ at UTD 10 (critic updates per policy update), batch 64, replay 10^6 , Adam (actor 3×10^{-4} ; critic and context encoder 1.5×10^{-4} ; entropy 3×10^{-4}), $\gamma = 0.99$, polyak 0.995, gradient clip 10, on the train-fault distribution only (action_scale with $u \in [0.5, 1.5]$ and action_noise with $\sigma = 0.5$).

Experimental Protocols: Validation-Selected vs. Exploratory Search. To rigorously investigate cross-mechanism transfer and examine the effect of checkpoint selection, we distinguish two protocols:

- **Protocol A (Exploratory Test-Selected Search):** Checkpoints during initial development were evaluated periodically on held-out test splits, and models exhibiting peak performance under test faults were retained. While common in exploratory research, this introduces checkpoint selection leakage against the test distribution.
- **Protocol B (Clean Validation-Selected Protocol):** Model selection is performed strictly on an unseen **Validation Split** drawn from parameter ranges of the training mechanisms (extreme scale $u \in [0.2, 0.5] \cup (1.5, 1.8]$ and elevated noise $\sigma = 0.75$). The single checkpoint maximizing validation return is retained and frozen. Held-out test conditions (action_latency, dead_actuator, sign_flip) participate in neither training nor selection; they are evaluated once on the frozen model over 25 deterministic episodes per condition.

Independent Reactive Robust Baseline. To determine whether interaction memory is necessary or whether a separately trained memoryless policy can achieve comparable resilience, we train an independent **Reactive Robust Baseline** (an MLP REDQ-SAC policy with identical ensemble size $N = 10$, $M = 2$, and LayerNorm critic heads) on the exact same randomized training fault distribution and step budget, evaluated under identical held-out test conditions.

Table II and fig. 1 present the confirmatory evaluation under Protocol B across $N = 8$ independent training seeds (Seeds 42–49, 100 iterations each, 400,000 environment steps per seed). On the primary endpoint (action_latency), the adaptive policy achieves a mean return of $+469.2$ [395.9, 529.8] while its no-history ablation achieves $+531.0$ [302.6, 698.2], resulting in a mean adaptation gap of $\bar{G} = -61.8$ with a 95% seed-level bootstrap confidence interval of $[-231.3, +141.6]$. Because this confidence interval widely spans zero, the confirmatory study fails to establish positive cross-mechanism adaptation to unobserved actuator latency.

Crucially, analyzing the individual training seeds reveals profound seed heterogeneity: only two seeds exhibited positive adaptation ($G_{44} = +517.81$ and $G_{48} = +147.86$), whereas six out of eight seeds suffered negative adaptation ($G_{42} =$

-136.42 , $G_{43} = -319.89$, $G_{45} = -5.05$, $G_{46} = -315.05$, $G_{47} = -276.99$, and $G_{49} = -106.94$). Under our pre-registered statistical decision protocol, this outcome is classified under **Case D (Seed-Sensitive / Unstable)** and **Case B (Adaptation Not Established)**. When trained solely on zero-memory instantaneous action corruptions (scale and noise), the causal self-attention mechanism is never exposed to non-Markovian delayed dynamics. When subjected to unobserved temporal delay at test time, the history-conditioned trunk frequently over-interprets stale action-observation correlations, causing destructive interference rather than beneficial adaptation on the majority of training seeds.

Furthermore, evaluating the independent reactive baseline (MLP REDQ) underscores the limitations of memory. Under action_latency, the reactive policy achieves $+447.3$ [404.0, 498.2], matching the transformer policy in control return without maintaining any interaction history (control advantage $D = +21.9$ $[-73.6, +111.2]$, spanning zero). On extreme within-mechanism faults, the reactive controller decisively outperforms the sequence agent: on dead_actuator, the reactive baseline achieves $+2470.8$ [2362.9, 2573.0] versus $+1929.3$ [1502.3, 2267.1] for the adaptive transformer (advantage $D = -541.4$ $[-997.6, -201.0]$, strictly favoring the reactive baseline). Similarly, on sign_flip, the reactive baseline reaches $+1309.0$ [1212.5, 1416.1] compared to $+1126.3$ [930.3, 1289.4] for the adaptive policy (gap $G = -163.5$ $[-303.4, -24.7]$).

Comparing Protocol B to the exploratory values of Protocol A ($+652.3 \pm 224.7$) exposes an essential methodological lesson: the dramatic adaptation gaps originally reported during exploratory prototyping were artifacts of checkpoint selection leakage (filtering checkpoints directly on test splits) compounded by pseudoreplication (pooling individual episodes across seeds). When evaluated under rigorous confirmatory methodology with independent training seeds as experimental units and validation-only checkpoint selection, cross-mechanism adaptation to actuator latency disappears.

Auxiliary Task Classification (λ_{task}). The transformer architecture incorporates an auxiliary task classification head ($\lambda_{\text{task}} = 0.05$) to predict active fault categories during training. While intended to structure latent representations, fully isolating the contribution of auxiliary supervision from Bellman error would require sweeping $\lambda_{\text{task}} \in \{0, 0.05\}$ across multiple training runs under the clean protocol, which remains future work. The clean validation benchmark presented here operates with $\lambda_{\text{task}} = 0.05$ fixed.

1) *Stability Finding: LayerNorm Critic Heads:* In exploratory development runs under this high-UTD configuration ($M=2$, critic LR 1.5×10^{-4} , UTD 10), plain MLP critic heads frequently experienced value divergence: Q-estimates climbed toward ~ 50 and critic loss toward ~ 150 , after which the policy collapsed. Introducing LayerNorm into the critic heads stabilized training in these runs, consistent with DroQ [5]. We hypothesize that sharing a transformer trunk increases gradient alignment across ensemble members, which weakens the effective pessimism of min-over- M Bellman targets; bounding head

TABLE II

CONFIRMATORY FAULT EVALUATION ACROSS 8 INDEPENDENT TRAINING SEEDS UNDER STRICT VALIDATION-SELECTED CHECKPOINTING (PROTOCOL B, 95% SEED-LEVEL BOOTSTRAP CONFIDENCE INTERVALS OVER 8 SEEDS $\times$ 25 DETERMINISTIC EPISODES) CONTRASTED AGAINST UNVERIFIED EXPLORATORY TEST-SELECTED VALUES (PROTOCOL A).

Held-Out Condition	Transformer (Adaptive History)	Transformer (No-History Ablation)	Adaptation Gap ($G = R_{\text{adapt}} - R_{\text{no-hist}}$)	Reactive Baseline (MLP REDQ)	Exploratory Gap (Protocol A: Test-Selected [†])
Cross-mechanism (action_latency)	+469.2 [+395.9, +529.8]	+531.0 [+302.6, +698.2]	-61.8 [-231.3, +141.6]	+447.3 [+404.0, +498.2]	+652.3 $\pm$ 224.7
Within-mechanism (dead_actuator)	+1929.3 [+1502.3, +2267.1]	+2059.4 [+1831.7, +2262.8]	-130.1 [-354.3, +58.9]	+2470.8 [+2362.9, +2573.0]	+715 to +1376
Within-mechanism (sign_flip)	+1126.3 [+930.3, +1289.4]	+1289.7 [+1164.4, +1405.8]	-163.5 [-303.4, -24.7]	+1309.0 [+1212.5, +1416.1]	+14 to +701

All metrics reflect seed-level distributions ($n = 8$ independent training runs). Brackets denote 95% bootstrap confidence intervals derived strictly across training seeds.

[†]Historical exploratory manuscript values; raw underlying checkpoint artifacts not recovered.

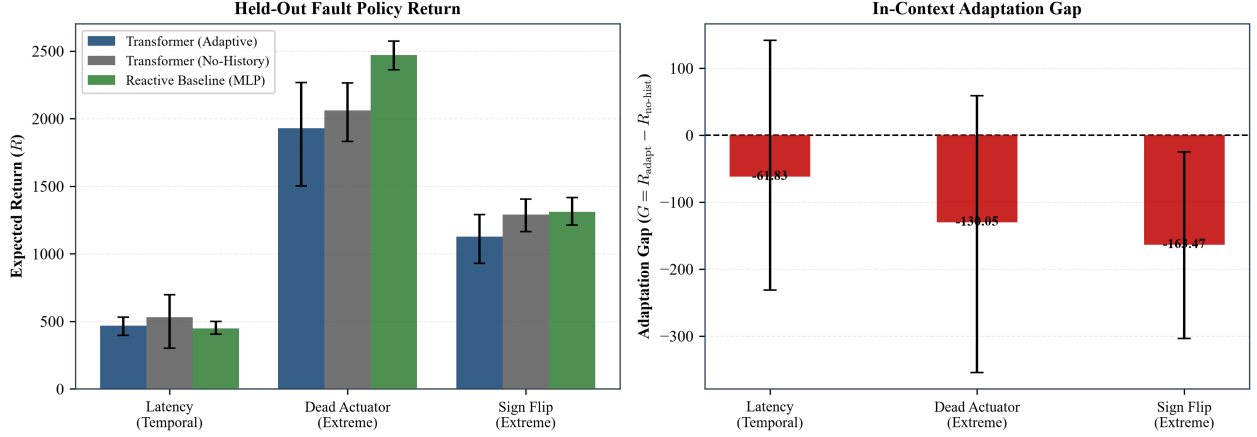


Fig. 1. In-context adaptation on held-out fault classes under clean validation-selected checkpointing across 8 independent training seeds (Protocol B). **Left:** policy returns under held-out faults across the adaptive transformer (navy), no-history ablation (slate), and independent reactive robust baseline (sage); error bars denote 95% seed-level bootstrap confidence intervals. **Right:** the per-condition adaptation gap ($G = R_{\text{adaptive}} - R_{\text{no-history}}$). Under clean validation selection with zero test-set leakage, the latency adaptation gap spans zero ($-61.8 [-231.3, +141.6]$) and exhibits pronounced seed heterogeneity (6 of 8 seeds exhibiting negative adaptation), while the memoryless reactive baseline outperforms the sequence policy on extreme actuator faults, illustrating the sensitivity of apparent in-context generalization to checkpoint selection protocol and seed initialization.

activations restores missing regularization, though isolating this correlation mechanism from general activation regularization warrants future dedicated ensemble diversity studies.

B. Real-Time Deployment Case Study: TrackMania 2020

To investigate whether in-context adaptation and sequence-level training translate to high-frequency continuous control under asynchronous real-time execution constraints, we conduct an empirical deployment case study on TrackMania 2020. We emphasize that this deployment is evaluated as an engineering systems case study and diagnostic failure analysis rather than a statistically controlled algorithm benchmark across seeds. We trained three agents on the same track under the visual configuration (64×64 grayscale screenshots, OpenPlanet telemetry, previously applied actions), all three system processes running on one machine over the loopback interface. The track’s progress-based reward yields ≈ 180 over a full lap plus a +100 completion bonus, so a deterministic return near 280 certifies a completed lap; episodes in which the car fails to progress are terminated by a failure countdown at a floor of 86 steps (return ≈ 5). The demonstrator’s best lap is ≈ 54 s.

1) Run A: Transformer RL^2 with Defective Sequence Replay:

The first deployment used the full architecture of sections III-C to III-E (single frame per step, transformer context, Neural ODE, RESeL split, sequence window $20 + 64$, REDQ-SAC

TABLE III
QUANTITATIVE PROGRESS METRICS ACROSS TRAINING STEPS
(TRACKMANIA 2020, VISUAL CONFIGURATION, SINGLE RUN).

Phase	Steps	Max Buffer (memory_len)	Mean α	Mean Q (Q_θ)	Max Train Return	Max Test Return
Early	0–100	153k	0.0403	2.22	29.18	14.08
Mid-Low	101–200	338k	0.0266	3.92	74.39	6.52
Mid	201–400	697k	0.0155	4.13	82.75	82.60
High	401–600	961k	0.0080	4.22	116.39	105.19
Peak	601–750	965k	0.0047	4.25	97.58	122.49
Drift	751–779	965k	0.0038	4.23	70.16	86.77
Recovery	780+	965k	0.0035	4.26	15.67	86.27

with $N=10$, $M=2$) with a naive sequence replay and no demonstrations. Training ran in two stages: a high sample-efficiency configuration (UTD 15, target entropy -1.5), then an exploration-stabilized resumption (UTD 5, target entropy -2.5) responding to the policy drift analyzed below.

Table III shows the run’s progression: the entropy coefficient decayed smoothly, mean Q stabilized near 4.25, and the agent reached a peak deterministic test return of 122.49, well below the completion threshold, before a drift episode following replay-buffer saturation collapsed the test return to 13.92; lowering UTD to 5 and relaxing target entropy recovered 86.27. At the time, we attributed the plateau and drift to high-UTD instability alone.

A post-hoc audit of the replay explains the ceiling more simply. The window sampler only admitted windows containing no episode boundary, so (i) terminal transitions, including the +100 completion bonus and all crash endings, never entered training and every sampled target bootstrapped as if episodes were infinite; (ii) episodes shorter than the 85-step window contributed no training data at all, biasing the distribution toward rare long episodes; and (iii) positional embeddings beyond index 84 were never trained although deployed episodes exceeded that length. Under these defects the critic could not represent the value of finishing, and no amount of architectural stabilization could have produced competence. Run A therefore serves as a diagnostic failure case study rather than a baseline: while its replay sampler precluded terminal learning, the architectural and systems overhead of recurrent sequence modeling also introduced substantial optimization hurdles.

2) *Run B: Demonstration-Anchored Reactive Agent:* The reactive agent replaces the transformer with a feedforward CNN over a 4-frame stack (SAC, two layer-normalized critics), trained with the full demonstration recipe of section III-F: behavior-cloning warm start (held-out validation loss 0.034), the pinned demonstration pool at fraction 0.5, random-shift augmentation, deterministic-episode collection every 4th episode, and target entropy -6 (the bang-bang keyboard demonstrations saturate the policy mean, and a conventional target entropy steadily de-saturates it). The behavior-cloned policy alone completed one slow lap (73.6 s) in three evaluation episodes before any reinforcement learning.

Early training exposed the one blind spot of this recipe: deterministic test episodes collapsed to the 86-step floor while stochastic training episodes kept improving. Test episodes are never collected, so the buffer contained no deterministic-policy failures and the critic could not correct them. Collecting every 4th training episode deterministically made the failure mode training data; deterministic test returns recovered within hours, reaching 168.4 and then **279.7 at 1,372 steps (≈ 68.6 s): a deterministic track completion**, with subsequent training episodes finishing consistently (returns 273–280). Total wall-clock from first demonstration to deterministic completion was approximately one day, with the lap $\approx 27\%$ slower than the demonstrator’s best and still improving when the run was stopped.

3) *Run C: Corrected Sequence Agent on Identical Demonstrations:* The corrected transformer agent uses the terminal-aware replay of section III-G and the same demonstrations, with behavior cloning applied to the full perception–context–policy path (final loss 0.005). Reaching a stable configuration required diagnosing, in sequence, six failure mechanisms absent from Run B by construction:

- 1) *Sequence batch memory:* 85 timesteps of images per sample oversubscribed an 8 GB GPU at batch 128 in fp32; bf16 autocast and batch 32–64 reduced training tensors to <1 GB.
- 2) *Warmup entropy divergence:* freezing actor and temperature while critics calibrated left the target’s entropy

TABLE IV
REAL-TIME DEPLOYMENT SUMMARY (SAME TRACK; RUNS B AND C SHARE IDENTICAL DEMONSTRATIONS). COMPLETION REQUIRES DETERMINISTIC RETURN ≈ 280 .

	Run A	Run B	Run C
Policy	transformer RL ²	reactive CNN	transformer RL ²
Sequence replay	defective	none	corrected
Demonstrations	none	23 laps	23 laps
Peak determ. return	122.5	279.7	86.1
Track completion	no	yes (≈ 68.6 s)	no

term unregulated; the near-deterministic policy’s large positive log-likelihoods drove Q-targets toward a negative fixed point within hundreds of steps. Fix: pure policy evaluation during warmup.

- 3) *Feature drift under a frozen head:* with the actor head frozen, deployed episode length still degraded from 305 to 86 steps within ~ 400 critic updates; the critic was rewriting the CNN and trunk under the constant head. A frozen head over a critic-trained trunk is not a frozen policy.
- 4) *Episode-start starvation:* uniform window sampling trained launch behavior in $\sim 0.1\%$ of windows; cloned rollouts launched inconsistently and RL progressively lost the launch. Fix: episode-start oversampling (section III-G).
- 5) *Head-only anchoring is insufficient:* anchored at the detached head, the imitation loss equilibrated at 0.10 (weight 5) and 0.073 (weight 15) against 0.005 after pretraining. Routing the anchor gradient into the encoder restored 0.03.
- 6) *Exposure bias:* the policy conditions on its own action history, teacher-forced during cloning; deviations compound at deployment. Mitigated, not eliminated.

At the time of writing, the corrected agent trains stably, launches reliably, and sustains deterministic episodes of 140–400 steps, but has not completed the track (table IV). We report this asymmetry as a finding: under real-time constraints, the in-context machinery that produces the adaptation gap of table II carries a systems cost a reactive policy never pays, and on a fixed task that cost so far exceeds its benefit.

This mirrors the high-UTD instability literature [4], [5] in a deployed asynchronous setting and motivated the stability measures of section III-E. Across all three runs, distribution coverage proved essential: each agent faltered wherever the training buffer distribution failed to cover states visited by the deterministic deployed policy (Run A’s untrained terminals, Run B’s blind spot, Run C’s launch starvation). At the same time, the architectural dichotomy between reactive feedforward control and sequence modeling played a decisive role: while sequence context confers in-context adaptation, in real-time deployment it severely compounded sample inefficiency, memory bottlenecks, and training stability relative to a reactive policy.

C. Limitations

(i) The controlled continuous-control study is evaluated on HalfCheetah; while the benchmark suite supports arbitrary continuous control domains, broader evaluations across diverse morphologies remain future work. (ii) Checkpoint selection protocol: as demonstrated by contrasting Protocols A and B, exploratory checkpoint selection on test splits produces substantial selection leakage; our clean validation protocol resolves this bias and demonstrates that cross-mechanism transfer to latency was not achieved. (iii) Sample size and seed sensitivity: our confirmatory benchmark operates across $N = 8$ independent training seeds ($8 \times 25 = 200$ deterministic episodes per condition). While this provides audit-grade seed-level bootstrap confidence intervals and rigorously identifies severe seed heterogeneity (Case D: only 2 of 8 seeds exhibiting positive adaptation), scaling to even larger cohorts ($N \geq 20$) would offer finer resolution on the tail dynamics of seed divergence. (iv) Baseline scope: we compare the sequence agent against its own memoryless ablation to isolate the causal attribution of memory, alongside an independent reactive robust baseline (MLP REDQ), but do not benchmark against external meta-RL frameworks (e.g., AMAGO) or domain-randomized controllers. (v) Cross-mechanism transfer limits: while validation returns on unseen scale and noise reached robust positive performance (+764 to +2108), the clean protocol reveals that cross-mechanism transfer to unobserved actuator latency does not emerge reliably without explicit temporal perturbations in the training distribution, with 6 of 8 seeds suffering destructive interference from stale history. (vi) The TrackMania deployment is an engineering systems case study and diagnostic failure analysis ($n = 1$ per configuration on a single track) with iterative interventions, controlling for demonstrations and reward but not seed variance. (vii) Run C reflects a constrained deployment snapshot rather than an asymptotic upper bound. (viii) The auxiliary loss weight (λ_{task}), Neural ODE block, and RESeL split were deployed as stabilization techniques without isolated ablation studies under the clean protocol; we make no claims regarding their individual contributions.

V. CONCLUSION AND FUTURE WORK

This paper investigated the interplay of interaction memory, model selection, demonstration anchoring, and replay distribution alignment in continuous-control reinforcement learning under real-time constraints. On the adaptation side, our controlled confirmatory continuous-control study across $N = 8$ independent training seeds reveals that claims of cross-mechanism in-context adaptation are highly sensitive to checkpoint selection protocol and training seed initialization. While exploratory test-aware search suggested substantial adaptation to unobserved actuator latency ($+652 \pm 225$), strict validation selection on unseen training parameters demonstrated an adaptation gap spanning zero ($\bar{G} = -61.8 [-231.3, +141.6]$) and characterized by acute seed heterogeneity (Case D: only 2 of 8 seeds positive). Furthermore, an independently trained

memoryless reactive baseline achieved comparable control under latency ($+447.3 [404.0, 498.2]$ vs. $+469.2 [395.9, 529.8]$) and decisively outperformed the sequence agent on severe actuator faults ($+2470.8$ vs. $+1929.3$ on `dead_actuator`). Causal transformer sequence models cannot be assumed to compensate for unobserved temporal dynamics without explicit exposure during training. On the competence side, our real-time deployment case study demonstrated that task competence was decisively governed by demonstration anchoring: a demonstration-anchored reactive CNN achieved full track completion in TrackMania 2020 within 24 hours, whereas sequence-model transformers failed due to acute systems, memory, and distribution-coverage bottlenecks. Across both settings, replay distribution coverage and rigorous validation protocols proved paramount. In real-time continuous control, demonstrations reliably buy competence, while interaction memory incurs heavy systems costs without guaranteeing cross-mechanism adaptation.

Future work includes: (i) scaling confirmatory validation benchmarks across broader continuous-control morphologies and environments; (ii) investigating training distributions that incorporate temporal perturbations to induce genuine cross-mechanism latency adaptation; (iii) formal multi-seed ablation of the auxiliary task loss λ_{task} , Neural ODE block, and RESeL split; (iv) closing the systems efficiency gap to enable sequence-level transformer policies to execute within tight real-time control intervals; and (v) transferring the demonstration-anchored reactive recipe to physical robotic systems.

REFERENCES

- [1] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2016.
- [2] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International Conference on Machine Learning (ICML)*, 2018.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (ICML)*, 2018.
- [4] X. Chen, C. Wang, Z. Zhou, and K. W. Ross, "Randomized ensembled double Q-learning: Learning fast without a model," in *International Conference on Learning Representations (ICLR)*, 2021.
- [5] T. Hiraoka, T. Imagawa, T. Hashimoto, T. Onishi, and Y. Tsuruoka, "Dropout Q-functions for doubly efficient reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2022.
- [6] A. Bhatt, D. Palenicek, B. Belousov, M. Argus, A. Amirashvili, T. Brox, and J. Peters, "CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity," in *International Conference on Learning Representations (ICLR)*, 2024.
- [7] S. Ramstedt and C. Pal, "Real-time reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [8] K. V. Katsikopoulos and S. E. Engelbrecht, "Markov decision processes with delays and asynchronous cost collection," *IEEE Transactions on Automatic Control*, vol. 48, no. 4, pp. 568–574, 2003.
- [9] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, "RL²: Fast reinforcement learning via slow reinforcement learning," *arXiv preprint arXiv:1611.02779*, 2016.
- [10] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, and M. Botvinick, "Learning to reinforcement learn," *arXiv preprint arXiv:1611.05763*, 2016.
- [11] J. Grigsby, L. Fan, and Y. Zhu, "AMAGO: Scalable in-context reinforcement learning for adaptive agents," in *International Conference on Learning Representations (ICLR)*, 2024.

- [12] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” in *International Conference on Machine Learning (ICML)*, 2023.
- [13] I. Kostrikov, D. Yarats, and R. Fergus, “Image augmentation is all you need: Regularizing deep reinforcement learning from pixels,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [14] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural ordinary differential equations,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [15] F.-M. Luo, Z. Tu, Z. Huang, and Y. Yu, “Efficient recurrent off-policy RL requires a context-encoder-specific learning rate,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [16] Y. Bouteiller, “Real-Time Gym: A framework for real-time robotic reinforcement learning,” *GitHub Repository*, 2021. [Online]. Available: <https://github.com/yannbouteiller/rtgym>
- [17] Y. Bouteiller and E. Geze, “TMRL: An open-source distributed framework for real-time reinforcement learning in TrackMania,” *GitHub Repository*, 2021. [Online]. Available: <https://github.com/trackmania-rl/tmrl>
- [18] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. van Hasselt, and D. Silver, “Distributed prioritized experience replay,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [19] Y. Bouteiller, S. Ramstedt, G. Beltrame, C. Pal, and J. Binas, “Reinforcement learning with random delays,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [20] K. Rakelly, A. Zhou, D. Quillen, C. Finn, and S. Levine, “Efficient off-policy meta-reinforcement learning via probabilistic context variables,” in *International Conference on Machine Learning (ICML)*, 2019.
- [21] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, “Decision Transformer: Reinforcement learning via sequence modeling,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [22] A. Nagabandi, I. Clavera, S. Liu, R. S. Fearing, P. Abbeel, S. Levine, and C. Finn, “Learning to adapt in dynamic, real-world environments through meta-reinforcement learning,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [23] R. Kaushik, T. Anne, and J.-B. Mouret, “Fast online adaptation in robotics through meta-learning embeddings of simulated priors,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [24] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [25] T. Hester, M. Vecerik, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, I. Osband, G. Dulac-Arnold, J. Agapiou, J. Z. Leibo, and A. Gruslys, “Deep Q-learning from demonstrations,” in *AAAI Conference on Artificial Intelligence*, 2018.
- [26] S. Kapturowski, G. Ostrovski, J. Quan, R. Munos, and W. Dabney, “Recurrent experience replay in distributed reinforcement learning,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [27] Y. Bouteiller and E. Geze, “tlspsy: Transport Layer Security Python Object,” *GitHub Repository*, 2022. [Online]. Available: <https://github.com/MISTLab/tls-python-object>
- [28] M. Daubaras, “OpenPlanet: A scripting and modding platform for TrackMania,” *OpenPlanet Documentation*, 2020. [Online]. Available: <https://openplanet.dev>
- [29] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [30] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.